

EĞİTİM :

ADO.NET

Bölüm :

Veriye Erişim

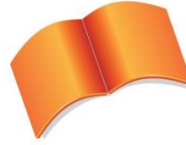
Teknolojileri & SQL Server

.Net Veri Sağlayıcısı

Konu :

Veri ve Veriye Erişim

Teknolojileri



Microsoft Türkiye

Açık Akademi

Veri ve Veriye Eriřim Teknolojileri

Birçok uygulama bazı bilgileri geici olarak tutup, daha sonra o bilgileri kullanarak işlemler yapar. Ancak bu bilgiler sadece uygulama alıştığı sürece erişilebilir durumdadır. Çünkü; bellek, uygulama alışırken bilgileri geici olarak tutmak için kullanılır. Bu şekilde alışan uygulamalara hesap makinası örnek olarak verilebilir. Yani kullanıcıdan birkaç veri alıp bunları hemen işleyen ve bu bilgilerin daha sonra kullanılması gerekmediği için geici olarak tutulmasında sakınca olmayan uygulamalardır.

Ancak her uygulama bu şekilde geliştirilemez. Çünkü alınan bilgilere uygulama kapatıldıktan sonra da erişilmesi gerekebilir. Bu durumda bilgileri bellek dışında, veriyi kalıcı olarak tutabilecek bir ortama, örneğin; disklere kaydetmek gerekecektir. İşte bu ihtiyacı karşılamak için farklı yöntemler geliştirilmiş ve günümüzde çok büyük miktarlarda veri tutan sistemler tasarlanmıştır.

Kısaca bu gelişime bakacak olursak; ilk olarak, günümüzde de halen çok sık kullanılan veri tutulabilecek belki de en basit **yapısal olmayan** sistemin “text dosya” olduğu söylenebilir. Ancak, bu yapılar da zamanla yetersiz kalmış ve **yapısal** fakat **hiyerarşik olmayan** çözümler geliştirilmiştir. Bu başlık altında Comma Separated Value (CSV) dosyalar, Microsoft Exchange dosyaları, Active Directory dosyaları örnek olarak verilebilir.

Zamanla, sadece veriyi tutmuş olmak da yetersiz kalmış ve verileri **hiyerarşik** bir yapıda tutma gereksinimi doğmuştur. Bu yapıya da verilecek en güzel örnek XML teknolojisidir. Bu yapının da yetersiz kaldığı durumlar için **ilişkisel (Relational)** veri depolama sistemleri geliştirilmiştir. İlişkisel veri depolama modeli günümüzde yoğun olarak kullanılmaktadır. Bu modelle saklanan verileri yönetmek için geliştirilen uygulamalara da ilişkisel veritabanı yönetim sistemi (Relational Database Management System, RDBMS) adı verilmektedir.

Böylece **veritabanını** (Database), veriyi daha sonra kullanılabilmesi ya da sadece depolanması amacıyla belli bir formatta tutan sistem olarak açıklayabiliriz.

Günümüzde büyük miktarlardaki verileri depolamak için ve bunları yönetmek için çeşitli firmaların geliştirmiş olduğu uygulamalar vardır. Bu uygulamalara örnek olarak **Microsoft SQL Server**, Oracle ve **Microsoft Access** verilebilir.

Şimdi veriye erişim teknolojilerinden bahsedilmesi gerekmektedir. Peki nelerdir bu teknolojiler?

Veriye Eriřim Teknolojileri

ODBC (Open Database Connectivity)

Microsoft ve diğer kuruluşların geliştirdiği bu teknoloji ile birçok veri kaynağına bağlanarak, veri alışverişi yapılabilmektedir. ODBC uygulama ortamlarına bir API sunmakta ve uygulamaların farklı sistemlere bağlanabilmesini sağlamaktadır.

ODBC teknolojisi 1990 öncesi geliştirilmiş olmasına rağmen ADO.NET platformunda da yer alan bir teknolojidir. ODBC hem yerel (local) hem de uzaktaki (Remote) veri kaynaklarına erişmek için kullanılacak bir veri erişim teknolojisidir.

DAO (Data Access Objects)

DAO, **ODBC**'nin ařađı seviye diller iin (C,C++) geliřtirilmiř olması ve kullanımının zor olması nedeniyle, Microsoft tarafından Visual Basic 3 ile geliřtirilmiř ve kullanımı ok kolay bir teknolojidir. Microsoft'un, **Microsoft Access** veritabanlarına eriřim standardı olan **Jet** iin geliřtirdiđi bu model, halen **VB6**'dan **MS Access**'e eriřim iin en hızlı yntemdir.

RDO (Remote Data Objects)

Microsoft'un **Visual Basic 4** ile duyurduđu **RDO**, **DAO**'nun **ODBC** sađlayıcısındaki eksikliđini gidermek iin geliřtirilmiřtir. Uzak veri kaynaklarına eriřimde **ODBC**'nin daha performanslı kullanılmasını sađlar.

OLE DB (Object Linking and Embedding DataBase)

ODBC'de olduđu gibi driver (sürücü) mantıđıyla alıřan **OLE DB**, arka tarafta **COM** arayüzünü kullanan bir tekniktir. Kaydettiđi geliřme ile bir ok sisteme bađlantı iin kullanılabilir. Bu bařarisından dolayı da **ADO.NET** ierisinde nemli bir yeri vardır.

ADO (ActiveX Data Objects)

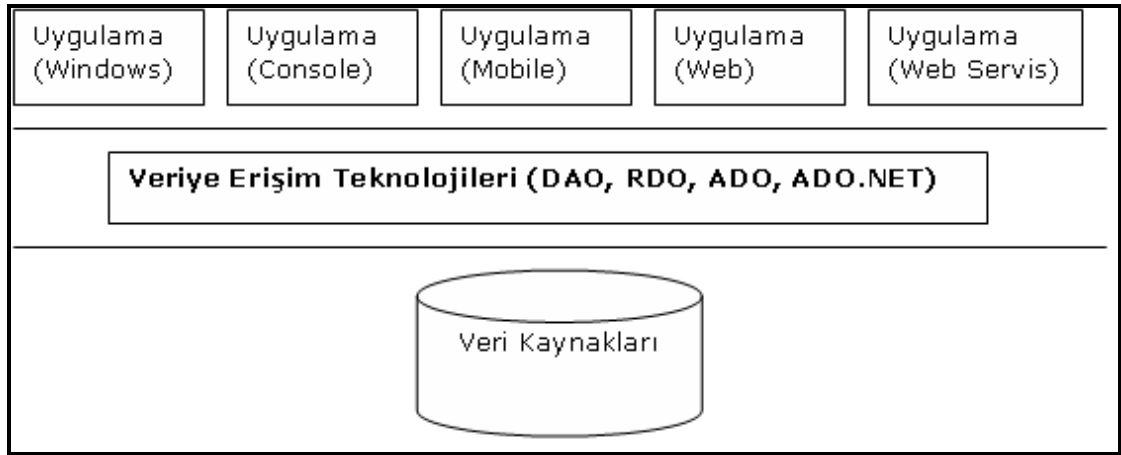
ADO aslında arka planda **OLE DB** teknolojisini kullanan ve veriye eriřimi daha da kolaylařtıran, yüksek seviye programlama dilleri iin veri eriřiminde tercih edilen bir teknolojidir.

ADO, ADO.NET'e temel oluřturduđu sylenen bir teknoloji olmasına rađmen bu tam olarak dođru deđildir. Bunun en byk nedenlerinden birisi de ADO'nun COM desteđine gereksinim duymasıdır. Gnmzde bazı platformlarda halen kullanılan bu teknoloji, XML standardına destek vermek iin eklentilerle glendirilmesine rađmen, XML formatındaki veriyi sadece tařıyabilmektedir. Aksine, ADO.NET, XML standardı zerine kurulmuř ve veriyi tařırken de XML formatını kullanan bir teknoloji olarak geliřtirilmiřtir.

ADO.NET

ADO.NET, Microsoft'un .NET platformunda ok nemli bir yere sahip, .NET uygulamalarından, her trl veri kaynađına eriřim iin gerekli hazır tipleri olan, COM desteđi gerektirmeyen, XML standardı zerine kurulmuř ve en nemlisi de .NET Framework'n imkanlarını kullanabilen, ADO'nun geliřmiř bir versiyonu olmaktan ok, yeni bir teknoloji olarak karřımıza ıkmaktadır.

ADO.NET, .NET Framework'n bir parası olarak 1.0'dan itibaren, .NET'in tm versiyonlarında yer almaktadır. Ayrıca veri iřlemlerini ok kolaylařtıran ve nesneye ynelik programlama (Object Oriented Programming) modeline uygun yapısı nedeniyle son derece kullanıřlı bir platformdur.



Veri kaynakları ve Uygulamalar arasındaki bağlantı

EĞİTİM :

ADO.NET

Bölüm :

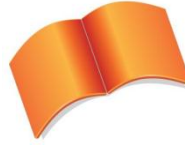
Veriye Erişim

Teknolojileri & SQL Server

.Net Veri Sağlayıcısı

Konu :

Veriye Erişim Yöntemleri



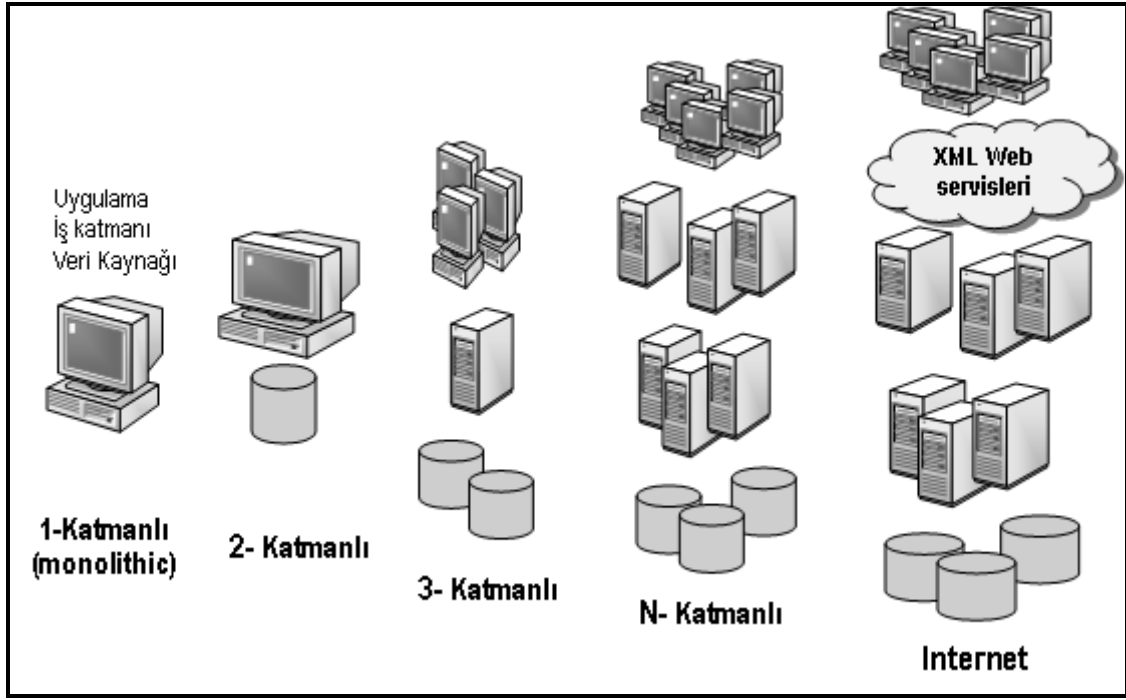
Microsoft Türkiye

Açık Akademi

Veriye Eriřim Yöntemleri

Veriye erişim yöntemleri uygulamalar tasarlanırken belirlenen stratejiler olarak karşımıza çıkmaktadır. Bu noktada, verinin nerede ve nasıl saklanacağı, kimlerin, bu veriler üzerinde, neleri yapmaya yetkili olacağı gibi konular erişim yöntemlerini seçerken önemli olmaktadır.

Veriye erişim yöntemleri uygulamayı katmanlara ayırmak olarak da düşünülebilir. Bir uygulamadaki mantıksal her birim bir **Katman** olarak isimlendirilir.



Katmanlı mimarideki gelişim süreci

Yukarıda da görüldüğü gibi, uygulamalarda zaman içerisinde, verinin tutulduğu ortam ile, verinin kullanıldığı ya da sunulduğu ortamın birbirinden ayrı olması gereksinimi ortaya çıkmış ve bu nedenle de katman sayısı giderek artmıştır. Tabi ki uygulamanın durumuna göre bu yapıların tümü kullanılabilir. Ancak burada önemli olan, uygulamanın şu anda nasıl çalıştığı değil, ilerleyen dönemlerdeki gelişmelere kolay entegre olabilecek, farklı platformlarda en ufak bir değişiklik dahi yapmadan kullanılacak bir sistem olmasıdır. Bu durumda şöyle bir öneride bulunmak doğru olacaktır; en azından veriye erişim katmanı ile, uygulama katmanının ayrılması ve hatta araya da en az bir katmanın dahil edilmesi, uygulamanın ölçeklenebilirliği ve daha sonra tekrar kullanılabilmesi adına güzel bir adım olacaktır.

EĞİTİM :

ADO.NET

Bölüm :

Veriye Erişim

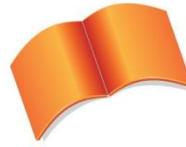
Teknolojileri & SQL Server

.Net Veri Sağlayıcısı

Konu :

ADO.NET Mimarisi ve

System.Data İsim Alanı

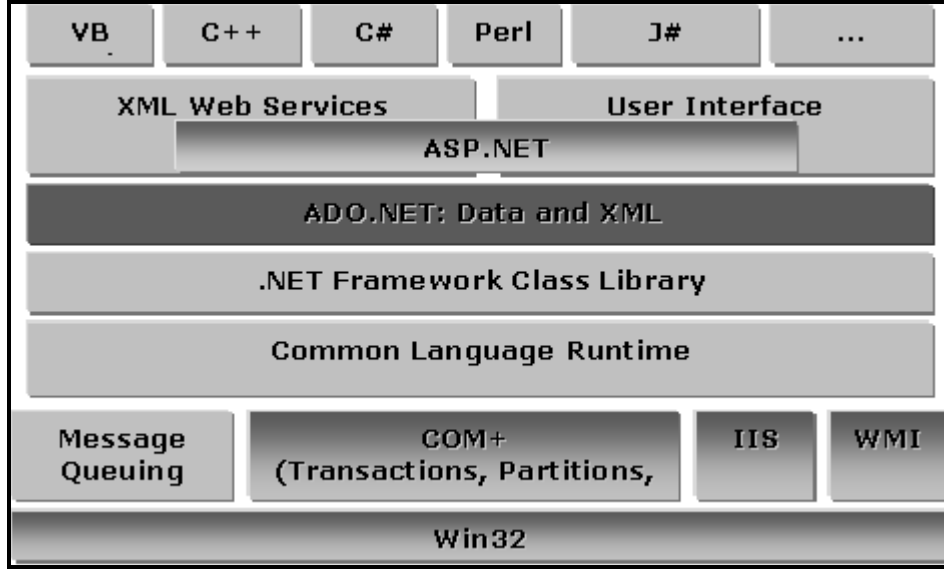


Microsoft Türkiye

Açık Akademi

ADO.NET Mimarisi

ADO.NET, .NET platformunda çok önemli bir noktada yer almaktadır. Bunu aşağıdaki şekilde net bir biçimde görebilmekteyiz.



ADO.NET'in .NET Framework içindeki yeri

Şekilde de görüldüğü gibi ADO.NET, .NET Framework'ün merkezinde ve bir çok platformda kullanılan ortak bir katmandır. O nedenle .NET ile geliştirilen tüm uygulamalar, veri kaynaklarına erişim için ADO.NET tiplerinden yararlanmaktadır. Bu da uygulama geliştiriciler için çok büyük bir avantaj sağlamaktadır. Çünkü ADO.NET platformu da, .NET Class Library üzerinde çalışmakta ve bu sayede .NET Framework içinde yazılmış olan bir çok tipi kullanabilmektedir. Bu tipler aracılığıyla, farklı veri kaynaklarına bağlanmak, verileri alıp, alınan verileri uygulamalarda kullanmak, yeni veriler eklemek ve mevcut verileri güncellemek çok kolay bir hal almaktadır.

Tabiki ADO.NET platformundan yararlanırken, .NET Framework'ün yapısından dolayı dikkat etmemiz gereken noktalar olacaktır. Çünkü ADO.NET ile, çok farklı veritabanı ve veritabanı yönetim sistemleri kullanılabilir. Böyle bir durumda farklı sistemlerle haberleşebilmek ve her biriyle en doğru, en performanslı şekilde çalışabilmek için farklı gereksinimler ortaya çıkmaktadır. Çünkü her sistemin kendi standartları ve desteklediği farklı standartlar vardır. İşte bu durumu göz önüne alarak .NET geliştiricileri, farklı standartları destekleyen tipler yazmışlar ve bunları ADO.NET kütüphanesinde ayrı ayrı isim alanları (Namespace) içerisine yerleştirmişlerdir. Bu isim alanlarından, SQL Server'a bağlantı kurmak için yazılmış tipleri bulunduran isim alanına **Sql Server Veri Sağlayıcısı (Sql Server .NET Data Provider)**, Oracle için olan isim alanına **Oracle Data Provider**, OleDb standartlarını destekleyen sistemlerle iletişim kurmayı sağlayan tiplerin yer aldığı isim alanına **OleDb .NET Data Provider**, ODBC standardı için olanlara da **ODBC .NET Data Provider** denilmektedir. Bu isim alanlarının tamamı, **System.Data** isim alanı altında yer almaktadır.

System.Data

ADO.NET ile uygulama geliştirirken kullanılabilecek, tüm veri sağlayıcılar için ortak olan bileşenlerin bulunduğu isim alanıdır. Mesela burada; çok güçlü ve çok da kullanışlı olan **DataSet**, **DataTable** gibi tipler vardır.

System.Data.SqlClient

SQL Server ile çalışabilmek için yazılmış tiplerin yer aldığı bu isim alanı, SQL Server 7.0 ve sonraki versiyonları ile birlikte kullanılabilmektedir.

System.Data.OleDb

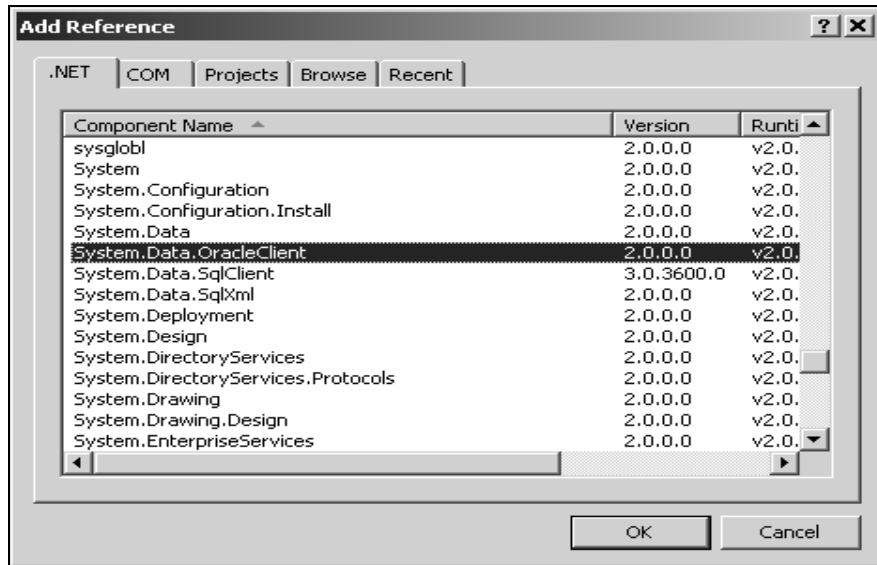
SQL Server 6.5 ve önceki sürümleri, Microsoft Access, Oracle, Text Dosyalar, Excel Dosyaları vb. gibi, OleDb arayüzü sağlayan tüm sistemlere bağlantı kurmak için gerekli olan tipleri barındırmaktadır. Ayrıca SQL Server'ın 6.5 sonrası versiyonları için de kullanılabilmektedir.

System.Data.Odbc

ODBC (Open DataBase Connectivity) standartlarını destekleyen ve ODBC sürücüsü bulunan sistemlere bağlantı kurmayı sağlayan tipleri içermektedir.

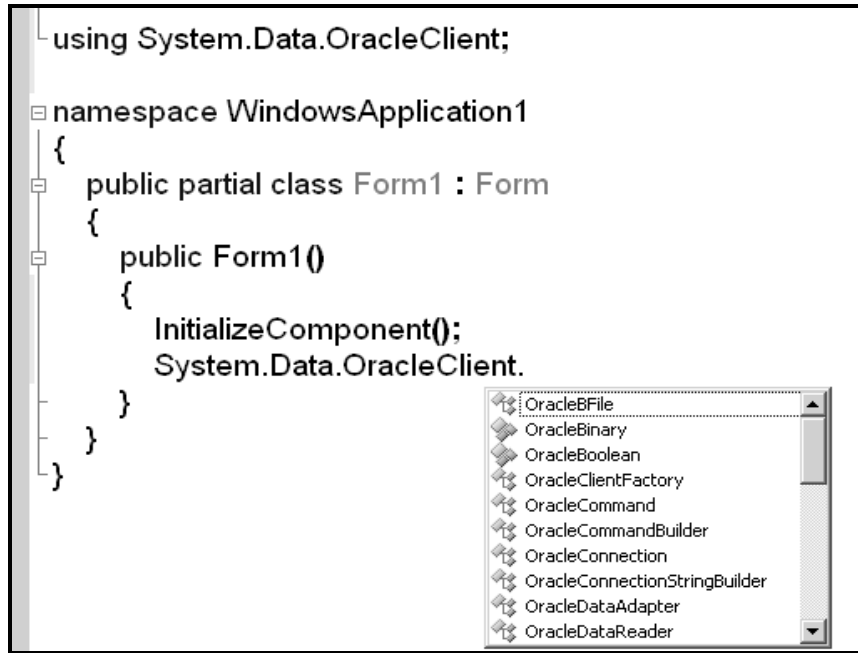
System.Data.Oracle

.NET 2.0 ile birlikte .NET Framework içerisinde Oracle'a biraz daha güçlü bir destek gelmiştir. Ancak Oracle veri sağlayıcısını kullanabilmek için öncelikle aşağıda görülen, **System.Data.OracleClient** referansının projeye dahil edilmesi gerekir.



Oracle Provider Referansı

Bu referansı ekledikten sonra, uygulamalarda Oracle için yazılmış olan tipler kullanılabilir. Bu tiplerin bazıları aşağıda görülmektedir. Aslında Oracle veri tabanlarına bağlanmak ile SQL Server veri tabanlarına bağlanıp veri almak, veriyi kullanmak ve hatta uygulama ortamındaki güncellemeleri veri kaynağına doğru gönderme işlemleri genel olarak aynı mantık üzerinde ilerlemektedir. Çünkü her ikisi de ve hatta diğer veri sağlayıcılar da aynı platformu kullandıkları için sadece nereye, hangi makineye, hangi veritabanına, hangi güvenlik ayarlarıyla bağlanması gerektiği gibi bilgilerde değişiklik yapılması gerekmektedir. Bu nedenle, eğitimin ilerleyen bölümlerinde yapılacak örnekler ve değinilecek konular **Sql Server .NET Data Provider** üzerinden işlenecektir.



Oracle Provider ile Kullanılabilecek Sınıflar

Sql Server .NET Data Provider bileşenlerini incelemeyi önce, herhangi bir veri kaynağı ile uygulamalar arasında veri taşıyabilmek için neler yapılması gerektiğini, provider'lardan bağımsız olarak incelemek, genel olarak ADO.NET mimarisine yukarıdan bakabilmek anlamına gelmektedir. Öncelikle en az bir adet **veri kaynağı** olması gerekmektedir. Burada veri kaynağı olarak Sql Server, Oracle, DB2, Interbase, Paradox, XML, Excel, Dbase, Access, CSV , Text vb... kullanılabilir.



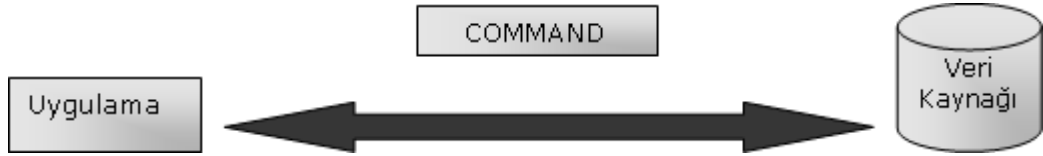
Veri Kaynağı (Data Source)

Veri kaynağı ile uygulamalar farklı makinalarda olabileceği gibi, aynı makina üzerinde de olabilirler, ancak nerede olurlarsa olsunlar, sonuçta iki farklı platformdan bahsedildiği için; uygulama ile veri kaynağını konuşturabilmek ve onları iletişime hazırlamak gerekecektir. İşte bu noktada, veri kaynağı ile uygulamaları konuşturacak olan, hangi makinedeki, hangi veri kaynağına hangi güvenlik ayarlarıyla gidilmesi gerektiği bilgilerini taşıyan **bağlantı (Connection)** nesnesine ihtiyaç duyulacaktır.



Veri Kaynağı ve Uygulama (Application)

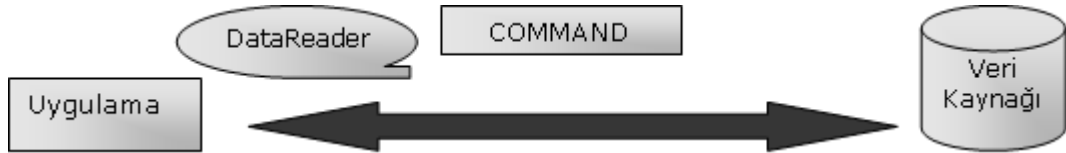
Veri kaynağı ile bağlantının kurulmuş olması, veri taşımak için yeterli olmayacaktır. Bir veri kaynağına veri göndermek için ya da oradan veri almak için bazı komutlara (Sql Cümleleri, Stored Procedure (Saklı yordam) isimleri) ve hatta bu komutlarla birlikte taşınması gereken parametrelere ihtiyaç olacaktır. ADO.NET platformunda bu işlemleri yapabilmeyi sağlayan ve belli bir bağlantı üzerinden ilgili veri kaynağına müdahale edebilen, oraya veri gönderme veya oradan veri alma isteğini sistemler arasında taşıyan **Command** adı verilen bir bileşen yer almaktadır.



Veri Kaynağı, Uygulama ve Command Nesnesi

Command adı verilen bileşen aracılığıyla, uygulama tarafından veri kaynağına veri gönderebilir. Bunun sonucunda herhangi bir işlem yapılmasına gerek yoktur. Ancak kullanılan sql cümlesi veri getirecek şekilde yazılmışsa, gelecek olan veriyi uygulama tarafında kullanabilmek için de bazı bileşenlere ihtiyaç olacaktır.

Bu bileşenlerden bir tanesi, yine provider'a göre değişen özelliklere sahip, verinin uygulamaya ileri yönlü ve sadece okunabilir olacak şekilde aktarılmasına imkan veren **DataReader** adındaki bileşendir.



Veri Kaynağı, Uygulama , Command Nesnesi ve DataReader

Yukarıda genel olarak bahsedilen bu bileşenlerin Sql Server .NET Data Provider isim alanındaki karşılıklarını ve en sık kullanılan ayrıntıları diğer bölümde ele alınacaktır.

EĞİTİM :

ADO.NET

Bölüm :

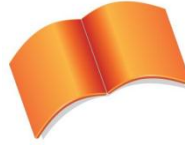
Veriye Erişim

Teknolojileri & SQL Server

.Net Veri Sağlayıcısı

Konu :

SqlConnection Nesnesi



Microsoft Türkiye

Açık Akademi

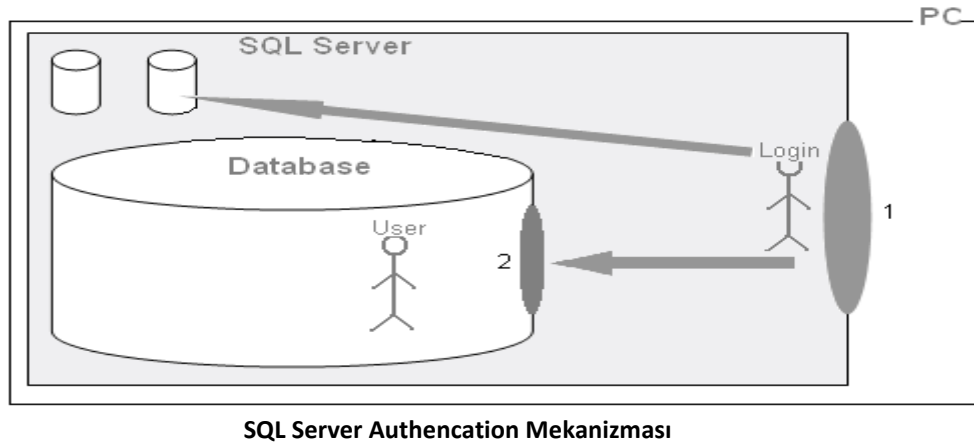
SqlConnection

Microsoft SQL Server 7.0 ve sonrası için kullanılan bağlantı sınıfıdır. Bu sınıfı kullanarak, bağlantı kurulmak istenen SQL Server örneğini, kullanılmak istenen veritabanını, ve bağlantı için gerekli olan güvenlik ayarlarını belirttikten sonra, bağlantı açılıp veritabanı kullanılabilir. Daha sonra diğer tipler devreye girer ve veri taşıma işlemleri başlatılabilir. Ancak unutulmaması gereken çok önemli nokta; açılan bağlantının işlemler bittikten sonra tekrar kapatılmasının gerekliliğidir. Eğer açılan bağlantılar kapatılmazsa, bir süre sonra SQL Server'ın kaynaklarının gereksiz yere harcanmasından dolayı sorunlar yaşanabilir.

SqlConnection sınıfı ile çalışırken dikkat edilmesi gereken belki de en önemli nokta bağlantı cümlesidir. Çünkü SQL Server'a bağlantı kurulurken, belli yetkilere sahip, belli rolleri olan kullanıcılar adına bağlantı kurulması gerekmektedir. Bu nedenle kısaca SQL Server'ın güvenlik modellerinden bahsetmekte yarar vardır.

SQL Server iki tip oturum açma modelini desteklemektedir. Bunlardan ilki; Windows'a giriş yapıldıktan (oturum açtıktan) sonra tekrar yeni bir şifre girmeyi gerektirmeyen **Windows Authentication** modelidir. Bu modelde, SQL Server'a bağlanacak kullanıcıların (**Login**) tekrar tekrar kontrol edilmesine gerek yoktur. Bir başka deyişle, işletim sistemine giriş yapabilen yani makinada oturum açma yetkisi olan kişilerin SQL Server'a giriş yapabildiği durumdur. Ancak SQL Server tarafından, Windows kullanıcılarının tanınıyor olmasına dikkat edilmelidir. Ardından bu kullanıcılar SQL Server'a bağlandıklarında, giriş yaptıktan sonra **nerelerde hangi işlemleri yapabilirler**, hangi işlemleri **yapamazlar** sorularına cevap aranmalıdır.

Bu noktada da odaklanılması gereken nokta kullanıcı (**user**) kavramıdır. Aşağıdaki şekilde login ve user kavramlarının geçerli oldukları alanlar belirtilmiştir.



Yukarıdaki şekilde görülen 1 kapısı, aslında SQL Server'a giriş yapılan kapıdır. Bu kapıdan geçebilmek için, SQL Server'da kayıtlı kullanıcı (Login) olunmalıdır. Giriş yapıldıktan sonra, SQL Server içerisine dahil olunur ancak herhangi bir veritabanında işlem yapma yetkisine sahip olunamaz. Bu durumda da işlem yapılacak ya da çalışılacak veritabanına ayrıca giriş yapılması gerekmektedir. Bunun için 2 kapısında gereken kontroller yapılır. 2 kapısındaki kontrolde, içeriye girmiş olan yani login olan kullanıcının ilgili veritabanında hangi isimle işlem yapacağı belirlenir. Aslında şöyle bir özet doğru olacaktır.

“SQL Server’a giriş yapabilmek için **Login** olarak kayıtlı olmak gerekir. Login olarak giriş yapıldıktan sonra da herhangi bir veritabanı üzerinde işlem yapabilmek için içeriye giriş yapan o Login’i ilgili veritabanı içerisinde temsil edecek olan **User** olarak veritabanı içerisinde tanıtmak gerekir.”

Bu bilgilerden sonra artık SQL Server’a kayıtlı Login’lerin SQL Server içerisine nasıl giriş yapabilecekleri noktalarına değinilebilir.

SqlConnection tipinin en çok kullanılan özellikleri ve metodları şunlardır.

ConnectionString	<i>“Data Source=Server; Initial Catalog=Veritabanı; Integrated security=True;Persist Security Info=False”</i> <i>“Data Source=Server; Initial Catalog=Veritabanı; Integrated security=False;Persist Security Info=False;User ID=loginadi; Password=sifre”</i> <i>“Data Source=Server; Initial Catalog=Veritabanı; Integrated security=True;Persist Security Info=False;Pooling=True/False; Max Pool Size=100; Min Pool Size=0 ”</i> Data Source : SQL Server instance adı. Initial Catalog : Veritabanı adı. Integrated Security : True; Windows Authentication ile bağlantının yapılması, False ise; SQL Server Authentication ile bağlantının yapılmasıdır ve UserID-Password girilmelidir. Persist Security Info : SQL Server’ın güvenlik bilgilerini uygulama tarafına geriye göndermesidir. Varsayılan olarak False değerine sahiptir. Pooling : SQL Server bağlantı havuzunu destekler. Aynı ConnectionString isteklerinde bağlantının baştan oluşturulması için tekrar kaynak harcamak yerine, mevcut bağlantının işlemler bittikten sonra havuza atılmasını sağlamak için kullanılır. Varsayılan olarak True değerine sahiptir. Max Pool Size : Aynı bağlantı cümleleri için açılan havuzda, maksimum kaç tane bağlantının bulunabileceğini belirtir. Min Pool Size : Aynı bağlantı cümleleri için açılan havuzda, minimum kaç tane bağlantının bulunması gerektiğini belirtir.
Open()	Bağlantının kullanıma hazır olması ve açılmasını sağlar. SQL Server üzerinden kaynakların ayrılması ve kullanılabilmesini sağlayan metottur.
Close()	Bağlantının kapatılmasını sağlar. SQL Server tarafından ayrılan kaynakların bırakılmasını sağlar.
ConnectionState	Bağlantının durumunu verir. Alabileceği değerler

	ConnectionState.  şeklindedir. Böylece bağlantının durumuna göre yapılacak işleme karar verilebilir.	
--	---	--

SqlConnection tipinin genel üyeleri

Örnek bağlantı cümleleri aşağıdadır.

/*Aşağıda oluşturulan bağlantı nesnelerinin herbirinde Data Source niteliğine localhost değeri atanarak, varsayılan Sql sunucusuna bağlanılacağı ifade edilmiştir. Şartlara göre localhost yerine, sunucu adı, ip numarası, sunucuAdı\örnekAdi gibi bilgiler girilmeside gerekebilir.

Initial catalog niteliği ise; bağlanılmak istenen sunucudaki hangi veritabanına bağlanılacağı bilgisini tutmaktadır.

Integrated Security niteliğine true, false veya SSPI değerlerinin atanması halinde ise; yukarıda şekilde de anlatılan SQL Server'a bağlantı kurulurken gerekli olan güvenlik bilgileri belirtilmiş olur. Integrated Security değerinin true ya da SSPI olması halinde, SQL Server'ın Windows Authentication modunu desteklemesi ve işletim sisteminde kayıtlı user'ın SQL Server'a da kayıtlı olması gerekmektedir.*/*

```
SqlConnection baglanti = new SqlConnection("Data Source=localhost;
Initial Catalog=ADONETDB; Integrated security=True");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=localhost;
Initial Catalog=ADONETDB; Integrated security=SSPI");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial
Catalog=ADONETDB; Integrated security=True");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial
Catalog=ADONETDB; Integrated security=SSPI");
```

```
SqlConnection baglanti = new SqlConnection("Data
Source=TCM\\SQLEXPRESS; Initial Catalog=ADONETDB; Integrated
security=True");
```

```
SqlConnection baglanti = new SqlConnection("Data
Source=TCM\\SQLEXPRESS; Initial Catalog=ADONETDB; Integrated
security=SSPI");
```

```
SqlConnection baglanti = new SqlConnection("Data
Source=192.168.1.2\\SQLINSTANCE1; Initial Catalog=ADONETDB; Integrated
security=True");
```

```
SqlConnection baglanti = new SqlConnection("Data
```

```
Source=192.168.1.2\\SQLINSTANCE1; Initial Catalog=ADONETDB; Integrated security=SSPI");
```

/* Aşağıdaki bağlantı cümlelerinde ise;
* Güvenlik bilgilerinin farklı olduğu durumlar görülmektedir. Bu cümlelerde, SQL Server'a windows Authentication modu ile değil, SQL Server'da açılmış kullanıcı hesapları ile nasıl oturum açılacağı gösterilmektedir.

* Aşağıdaki cümlelerde güvenlik bilgileri yukarıdakilere göre farklıdır. windows Authentication modunun kullanılmayacağı Integrated Security=False ifadesi ile belirtilmektedir. User ID ve Password bilgileri girildiği için; Integrated Security niteliği otomatik olarak false değerini alır. */

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial Catalog=ADONETDB; Integrated Security=False; User ID=sa; Password=1234");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial Catalog=ADONETDB; Integrated Security=False; User ID=tcmllogin; Password=1234");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial Catalog=ADONETDB; User ID=tcmllogin; Password=1234");
```

```
SqlConnection baglanti = new SqlConnection("Data Source=.; Initial Catalog=ADONETDB; User ID=sa; Password=1234");
```

Bu bilgiler ışığında bir örnek ele alıp, veritabanına bağlantı kurup, bağlantının veri taşınabilmesi için hazırlanmasını sağlayan işlemleri inceleyelim.

Örnek Uygulama

Bu örnekte, SQL Server'a bağlantı kurup, bu bağlantıyı açarak, veritabanını kullanıma hazır hale getireceğiz. SqlConnection nesne örneğinin Open() metodu çağırıldıktan sonra, 5 saniye boyunca ilgili bağlantı açık tutulmakta ve sonrasında bağlantı kapatılmakta, böylece sistem kaynakları gereksiz yere tüketilmemektedir. Burada genel bir kuralı belirtmekte fayda var: Çoğunlukla söz konusu veri kaynaklarına olan bağlantılar mümkün olduğunca geç açılmalı ve işlemleri takiben mümkün olduğunca erken kapatılmalıdır.

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Data.SqlClient;

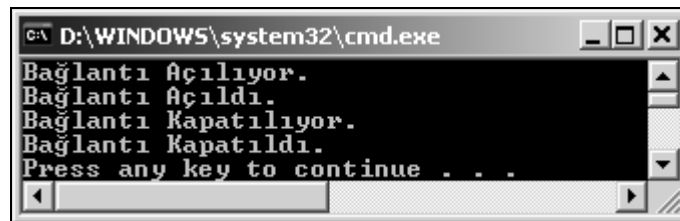
namespace VeriTabanıBaglantısı
{
    class Program
    {
        static void Main(string[] args)
        {
            SqlConnection baglanti = new SqlConnection();

            //baglanti nesnesini oluştururken, yapıcı metodun (Constructor) aşırı yüklenmiş versiyonları da kullanılabilir.
            baglanti.ConnectionString = "Data Source=.;Initial Catalog=SDS; Integrated Security=True";
```



```
//bağlantı açılır. Böylece veritabanı kullanıma hazır hale  
gelmiş olur.  
    Console.WriteLine("Bağlantı Açılıyor.");  
    baglanti.Open();  
    Console.WriteLine("Bağlantı Açıldı.");  
  
    System.Threading.Thread.Sleep(5000);  
  
    Console.WriteLine("Bağlantı Kapatılıyor.");  
    baglanti.Close();  
    Console.WriteLine("Bağlantı Kapatıldı.");  
}  
}  
}
```

Bu kodların çalışması sonucu aşağıdaki ekran görüntüsü oluşmaktadır.



EĞİTİM :

ADO.NET

Bölüm :

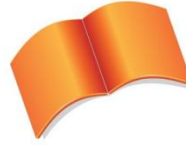
Veriye Erişim

Teknolojileri & SQL Server

.Net Veri Sağlayıcısı

Konu :

SqlCommand Nesnesi



Microsoft Türkiye

Açık Akademi

SqlCommand

Uygulama ile SQL Server arasında sql komut cümlelerinin, saklı yordam (stored procedure) isimlerinin, gerekli parametre isimleri ve değerlerinin taşınmasını sağlayan sınıftır. Taşıdığı sql komut cümlesini ya da çağıracağı saklı yordamın ismini, SQL Server tarafında yürütme ve çıkan sonuçları ya da etkilenen kayıt sayısını geriye gönderme özelliklerine de sahiptir. SqlCommand sınıfının en sık kullanılan özellik ve metodları aşağıda kısaca incelenmektedir.

CommandText	Parametrelili ya da parametresiz sql cümlelerinin, parametre bekleyen ya da beklemeyen saklı yordam isimlerinin atanabileceği özelliktir.
CommandType	CommandText özelliğinde belirtilen string ifadenin, saklı yordam olarak mı yoksa, Text ifade olarak mı değerlendirileceğini belirten özelliktir. Varsayılan olarak Text değerine sahiptir. Ancak saklı yordam (stored procedure) olarak kullanılacaksa; özellikle belirtilmesi gerekmektedir.
Connection	SqlCommand nesnesinin içerdiği sorguların hangi sunucu ve veritabanında çalıştırılması gerektiği ile ilgili bağlantı nesnesini tutan özelliktir.
ExecuteNonQuery()	Yazılan sql cümlesi ya da SQL Server tarafındaki saklı yordamın (stored procedure) çalıştırılması ve bu işlem ya da işlemler sonucunda etkilenen kayıt sayısının geriye alınabilmesini sağlayan metottur. Ancak burada dikkat edilmesi gereken nokta; geriye sadece tamsayı tipinde bir değer dönmektedir. Bu nedenle, genellikle veritabanına değer gönderirken yani, insert, update ve delete işlemleri için kullanılır. Sonrasında da bu işlemlerden etkilenen kayıt sayısı geriye alınmaktadır. Böylece örneğin güncellenen satır sayısı elde edilip kullanılabilir.
ExecuteScalar()	ExecuteNonQuery metodu gibi, yazılan sql cümleleri veya saklı yordamları çalıştırmak için kullanılır. Bu metodun farklı olduğu nokta ise; geriye sadece tek bir object tipinde sonuç dönen sorgular için kullanılmasıdır. Bu metotla, select işleminden dönecek olan scalar kayıtlar alınabilmektedir. Örneğin gruplama fonksiyonları içeren sorgu cümlelerinin sonuçlarını elde etmekte kullanılabilir.
ExecuteXmlReader()	Sadece SqlCommand sınıfında (class) bulunan bu metot; SQL Server üzerinden gelen verinin, XML (eXtensible Markup Language) formatında alınabilmesini sağlar.
ExecuteReader()	Geriye veri ya da veri blokları dönen sql cümleleri veya saklı yordamları çalıştıran ve bunların sonuçlarının uygulama tarafına alınabilmesini sağlayan metottur. Bu metot ile uygulama tarafına gelecek sonuçlar, DataReader adı verilen ve veri sağlayıcıya (data provider) göre değişen bir tip yardımıyla okunabilmektedir. Özellikle içeriği çok fazla değişmeyen ve az sayıda satır içeren veri kümelerinin uygulama ortamına daha hızlı bir şekilde alınabilmesini sağlayan modellerde tercih edilmektedir.

SqlCommand tipinin genel üyeleri

Örnek SqlCommand kullanımları aşağıdadır.

```
SqlCommand komut = new SqlCommand();
komut.CommandText = "Insert INTO Tablo(Alan1,Alan2,
Alan3) VALUES (deger1,deger2,deger3)";
komut.Connection = baglanti;
komut.CommandType = CommandType.Text;
baglanti.Open();
int etkilenenKayitSayisi = komut.ExecuteNonQuery();
baglanti.Close();

SqlCommand cmd = new SqlCommand();
cmd.CommandText = "UPDATE tablo SET alan1=deger1, alan2
=deger2 where alan=deger";
cmd.Connection=baglanti;
//commandtype belirtilmek zorunda değildir.
baglanti.Open();
cmd.ExecuteNonQuery();
baglanti.Close();

SqlCommand cmd = new SqlCommand();
cmd.CommandText = "select count(*) from tablo";
cmd.CommandType = CommandType.Text;
cmd.Connection = baglanti;
baglanti.Open();
int kayitsayisi = (int)cmd.ExecuteScalar();
baglanti.Close();

SqlCommand cmd = new SqlCommand();
cmd.CommandText = "SP_TumVerileriGetir";
cmd.CommandType = CommandType.StoredProcedure;
cmd.Connection = baglanti;
baglanti.Open();
SqlDataReader rdr = cmd.ExecuteReader();
baglanti.Close();
```

SqlCommand Kullanım Örnekleri

EĞİTİM :

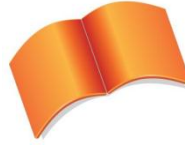
ADO.NET

Bölüm :

**Bağlantılı (CONNECTED)
Model & Bağlantısız
(DISCONNECTED) Model**

Konu :

SqlDataReader Nesnesi



Microsoft Türkiye

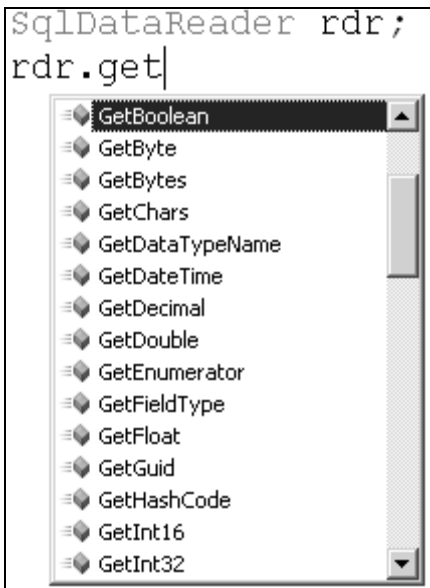
Açık Akademi

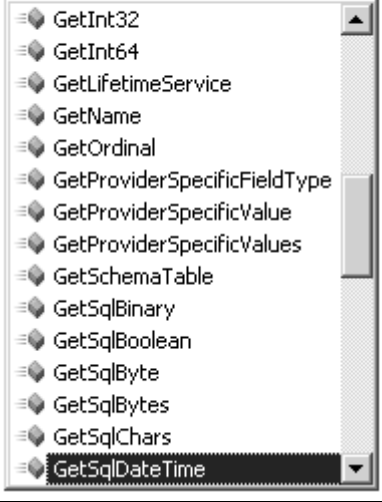
SqlDataReader

SQL Server üzerinde veya herhangi bir veri kaynağında duran veriler, veri sağlayıcıya bağlı connection ve command nesneleri yardımıyla, uygulama ortamına alınabilmektedir. Ancak veri kaynağından gelen bu veriyi uygulama tarafında kullanabilmek için, bir şekilde o veriyi bellekte tutabilecek ve uygulamada ihtiyaç olduğu anda kullanılmasını sağlayacak başka bir bileşene, nesneye ihtiyaç vardır. Bu ihtiyacı karşılayan, veri sağlayıcıya özel **DataReader** sınıfları bulunmaktadır. Sql Server .NET Data Provider için bu sınıf, **SqlDataReader** olarak kullanılmaktadır.

SqlDataReader sınıfından oluşturulan değişken, **SqlCommand**'ın **ExecuteReader()** metodundan dönen SqlDataReader nesne örneğini yakalar. Bu nesne örneği, uygulamalarda yazılan sql cümlesine bağlı olarak, gelen sonuç kümesini (result set) **sadece ileri yönlü (forward only) ve yalnız okunabilir (read only)** formatta kullanma şansı verir. Bu durum, verinin daha hızlı bir şekilde uygulama ortamına alınmasını sağlar.

SqlDataReader'ın kullanımı sırasında dikkat edilmesi gereken önemli noktalar vardır. Örneğin; bir SqlDataReader, bir SqlConnection nesnesini kendine bağlar ve kayıtlar üzerindeki işin sonuna gelinmeden bağlantının serbest kalmasına izin vermez. Ayrıca bir SqlDataReader, üzerinden veri taşıdığı bağlantı nesnesini tamamen kendi kullanımına alır ve kapatılmadan önce bağlantı nesnesini serbest bırakmaz. Ancak bu soruna .NET 2.0'dan itibaren, MARS (Multiple Active Result Sets) adı verilen teknoloji ile çözüm bulunmuştur. Ne var ki bu teknolojinin kullanılabilmesi için ilgili veritabanı sisteminin MARS'a destek veriyor olması gerekmektedir. Örneğin MS SQL Server 2005 ve sonraki sürümlerin doğrudan MARS desteği varken bu durum MS SQL Server 2000 ve önceki sürümleri için geçerli değildir. SqlDataReader'ın en sık kullanılan özellik ve metotları şunlardır:

Read()	SqlDataReader'ın en önemli metodudur. Reader'ın bir satır sonrasına konumlanmasını sağlar. Eğer bir sonraki satıra geçebilmişse, true değer döner. Eğer okunacak kayıt yoksa false değeri döner. Bu metot bir while döngüsünde etkin bir şekilde kullanılabilir. Böylece bir sonuç kümesi üzerindeki tüm satırlar tek tek dolaşılıp değerlendirilebilir.
Get....()	Get ile başlayan metotlar, reader'ın her Read() metodu çağrısı ile okuduğu satırdaki sütunların değerlerini ve aynı zamanda tiplerini de dönüştürerek alabilmeyi sağlar. 

	<pre>SqlDataReader rdr; rdr. </pre>  Ancak bu metotları kullanarak verilere erişmek bazı durumlarda tehlikeli olabilir. Bu metotlarla, veritabanı tarafında NULL olan değerleri okurken dikkatli olmak gerekir. Çünkü NULL değerler uygulamaya bu metotlarla alındığında uygulama tarafında çalışma zamanı istisnaları (run time exceptions) oluşmaktadır. Bu nedenle uygulama tarafına gelen verileri; <pre>while (rdr.Read()) { rdr["kolonadi"].ToString(); }</pre> ya da <pre>while (rdr.Read()) { rdr[kolonindeksnumarası].ToString(); }</pre> şeklinde almak hata oluşmasının önüne geçmeyi sağlar.
Close()	DataReader'ın close metodu, DataReader'ın kullandığı bağlantı nesnesini bırakmasını sağlar. Böylece bu bağlantı nesnesini başka bir DataReader nesne örneği kullanabilir.

SqlDataReader için genel üyeler

Buraya kadar bahsedilen SqlConnection, SqlCommand ve SqlDataReader sınıfları ile geliştirilen uygulamalar, **Bağlantılı (Connected)** veri erişim modeli olarak isimlendirilmektedir. Bağlantılı model olarak isimlendirilmesinin nedeni, **DataReader nesnesiyle veriyi sunarken, son veriye gelene kadar bağlantının açık olmasının zorunlu olmasıdır.**

Örnek Uygulama

Bu örnekle, bağlantılı modelde veritabanından veri nasıl alınır, veritabanına nasıl veri gönderilir, mevcut veriler nasıl güncellenir ya da nasıl silinir gibi konular işlenecek. Bu işlemleri yapmak için neler gerekli, neler yapılmalı, nelere ihtiyaç olacak gibi sorulara da yanıt verilecek.

İlk olarak veritabanı ve tablolar oluşturulmalıdır. Senaryo, KATILIMCI kayıtlarının tutulacağı basit bir sistem üzerinden ilerleyecektir. Bu nedenle SQL Server üzerinde, uygun bir veritabanı içerisinde, aşağıda görülen KATILIMCI tablosu oluşturulmaktadır.

	Column Name	Data Type	Allow Nulls
	KatID	int	☐
	KatIsim	nvarchar(50)	☐
	KatSoyIsim	nvarchar(50)	☐
	KatDogTar	datetime	☒
			☐

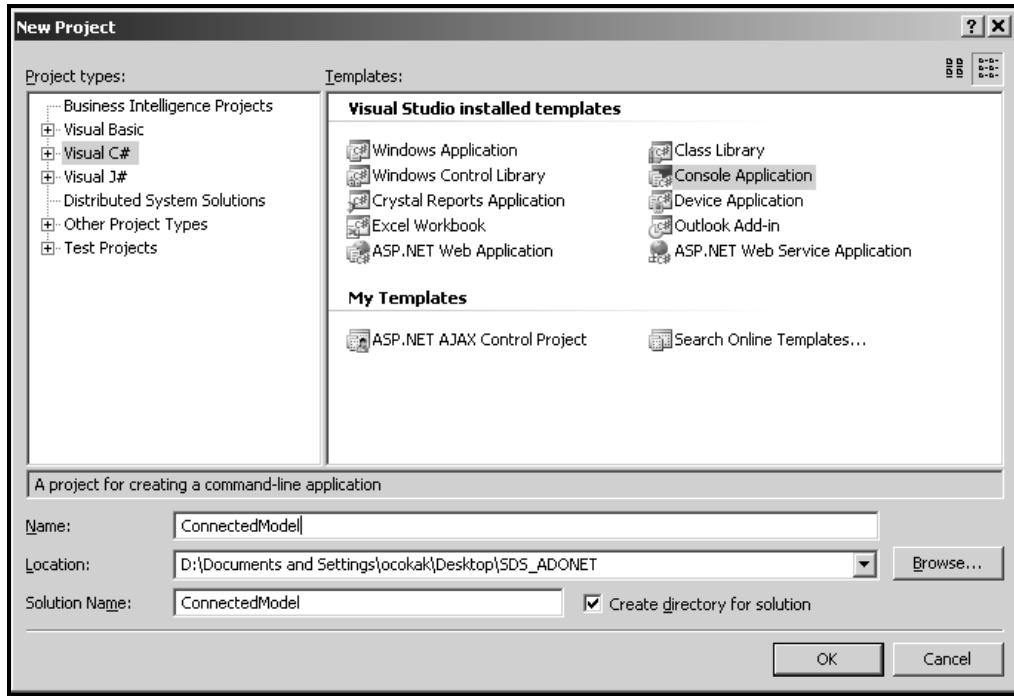
KATILIMCI tablosunun alanları

Burada KatID kolonu Primary Key ve Identity yani otomatik olarak artan bir kolondur. Ayrıca dikkat edilmesi gereken diğer bir nokta da KatDogTar adındaki alanın NULL değerler alabileceğidir. Bu nokta önemlidir çünkü uygulama tarafından bu alana değer gönderilme zorunluluğu olmamasına rağmen, diğer alanlar için aynı durum geçerli değildir. Tabloya birkaç test verisi girilerek kodlamaya geçilebilir.

	KatID	KatIsim	KatSoyIsim	KatDogTar
	1	Osman	ÇOKAKOĞLU	01.01.2007...
	2	Burak	ALTUNTAŞ	02.02.2006...
	3	Bülent	SÖZGE	03.03.2005...
	4	Özgür	ALTUNTAŞ	NULL
*	NULL	NULL	NULL	NULL

KATILIMCI tablosu için örnek veriler

Uygulama konsol uygulaması (Console Application) olarak açılacak ve öncelikle mevcut veriler uygulama tarafına alınacaktır.



Proje şablonunun seçimi

Öncelikle şunu belirtmek gerekir; Veritabanının SQL Server olması nedeniyle kullanılacak veri sağlayıcısı, **SQL Server .NET Data Provider**'dır. Demek ki kullanılacak tipler, **System.Data.SqlClient** isim alanı altında yer almaktadır. Bu durumda ilk olarak bu isim alanı **using** bloğu ile uygulamaya eklenmelidir. Bu bir zorunluluk olmamakla beraber, kod içerisinde ilgili isim alanındaki tiplere daha kolay ulaşılmasını sağlayacaktır ki bu da kodun okunabilirliğini artırır.

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Data.SqlClient;

namespace ConnectedModel
{
    class Program
    {
        static void Main(string[] args)
        {
            SqlConnection baglanti = new SqlConnection();
            //baglanti nesnesi oluşturulurken, yapıcı metodun aşırı
            //yüklenmiş versiyonları da kullanılabilir.

            baglanti.ConnectionString = "Data Source=.;Initial
            Catalog=SDS; Integrated Security=True";

            SqlCommand cmd = new SqlCommand();
            //Command nesnesi oluşturulurken, yapıcı metotlardan
            //(Constructor) yararlanılabilir.

            //cmd nesnesinin çalıştıracağı SQL cümlesi yazılır.
            cmd.CommandText = "SELECT * FROM KATILIMCI";

            //cmd nesnesinin kullanacağı yani gideceği yeri gösteren
            //baglanti nesnesi atanır.
            cmd.Connection = baglanti;
```

```

//bağlantı açılır. Böylece veritabanı kullanıma hazır
hale gelir.
    baglanti.open();
//burada dikkat edilemesi gereken önemli bir nokta
vardır. SqlDataReader sınıfının yapıcı metodu yoktur. Ancak kodlarken izin
varmış gibi görülür. Fakat uygulamayı build ederken hata alırız. Bunun
nedeni SqlDataReader sınıfına ait yapıcı metodun tanımlandığı assembly
dışında çağırılmamasıdır. Çünkü, internal erişim belirleyicisi ile
işaretlenmiştir.
    //SqlDataReader rdr = new SqlDataReader(); //HATALI

    SqlDataReader rdr;

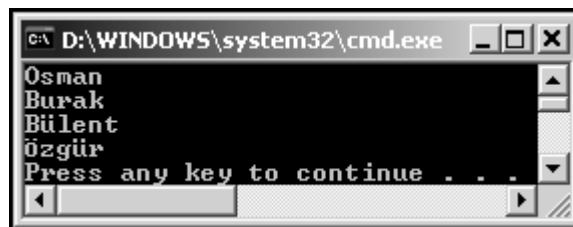
    //SqlDataReader nesne örneğine,Command'ın veritabanından
execute işlemi sonucu elde edilen sonuç kümesi bağlanır.
    rdr = cmd.ExecuteReader();

    //HasRows özelliği, SqlDataReader'ın gösterebileceği
satırı olup olmadığı bilgisini verir.
    if (rdr.HasRows)
    {
        //Read-only ve forward-only olarak alınan verilerin
hepsine erişebilmek için; baştan sona kadar birer birer ilerlemeyi
sağlayan while döngüsü hazırlanır
        while (rdr.Read())
        {
            //KatIsim alanının değerlerini sırasıyla ekrana
yazdırır.
            Console.WriteLine(rdr.GetString(1));
        }
    }

    //bağlantı kapatılıyor. Böylece veritabanı kullanıma
kapatılmış oluyor ya da başka kullanıcılar için hazır hale geliyor.
    baglanti.Close();
}
}
}

```

Bu şekilde uygulamayı çalıştırsak;



Tüm Katılımcıların SqlDataReader ile okunması

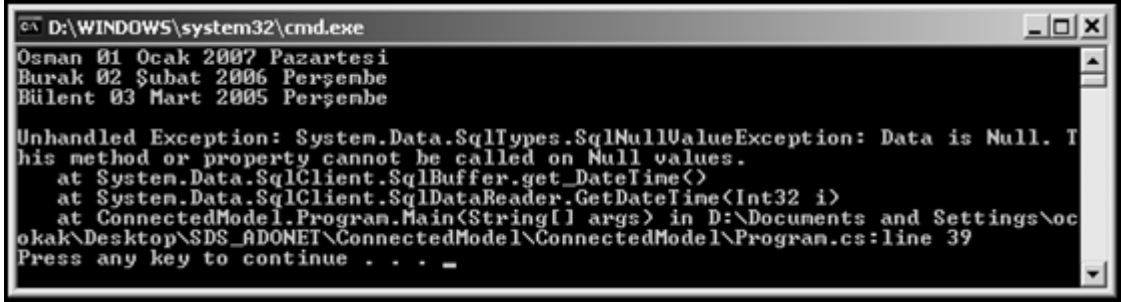
şeklinde bir çıktı alırız. Burada dikkat edilirse; **GetString()** metodu kullanılmıştır. Ancak herhangi bir hata ile karşılaşmamıştır. Çünkü KatIsim alanında NULL değer yoktur. Eğer KatDogTar alanının değeri bu şekilde alınmak istenirse;

```

while (rdr.Read())
{

```

```
//KatIsim alanı ve KatDogTar değerlerini sırasıyla ekrana yazdırır.
Console.WriteLine(rdr.GetString(1) + " " +
rdr.GetDateTime(3).ToLongDateString());
}
```



Null değer istisnası

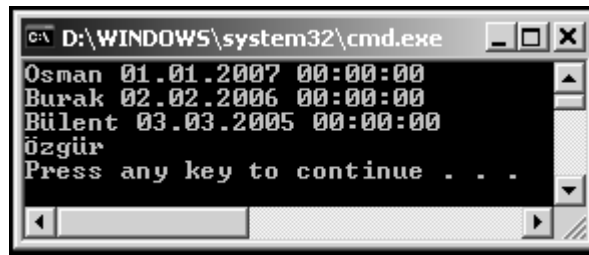
şeklinde bir hata mesajı alınır. Bu nedenle DataReader nesnesinden verileri alırken, her zaman bu olasılıklara karşı hazırlıklı olunmalıdır. Burada aslında yapılabilecek birkaç değişiklik vardır. Bunlardan bir tanesi,

rdr.IsDBNull(3)

metodu ile her değerın NULL olup olmama kontrolünü yapmaktır. Bu metod geriye Boolean bir değer dönmektedir. Diğer bir yol ise;

```
while (rdr.Read())
{
    //KatIsim alanı ve KatDogTar değerlerini sırasıyla ekrana yazdırır.
    Console.WriteLine(rdr[1].ToString() + " " +
rdr["KatDogTar"].ToString());
}
```

şeklinde kullanmaktır. Ancak burada da NULL olan değeri DateTime formatına dönüştürmeye çalışırken hata alınmaktadır. String olarak kullanıldığında ise sorun olmamaktadır.



Null değer ile mücadele

Burada dikkat edilirse, SqlDataReader nesnesinden verileri alırken aslında bir kaç yol olduğu görülebilmektedir.

rdr.Get....(indeks) → Bu şekilde, değeri alınmak istenen kolon indeksi verilebilir.

rdr[indeks] → Bu şekilde, değeri alınmak istenen kolon indeksi verilmektedir. Yukarıdaki kullanımdan farkı ise; NULL değerleri boş string değeri olarak almasıdır.

rd[“kolonadi”] → Bu şekilde, değeri alınmak istenen kolonun adı belirtilmektedir. Ayrıca ikinci kullanımdaki gibi NULL değerler, boş string ifade olarak gelmektedir.

Burada incelenmesi gereken diğer bir nokta da **while** döngüsüdür. while parantezleri içerisinde bakılan koşul ifadesi aslında DataReader’a ait Read() metodundan dönen Boolean değerdir. Bu değer, her while döngü bloğu işletiminde bir satır okumak isteyecektir. Eğer yeni bir satır okuyabilirse; bu satırı, while bloğu içinde **rd** ya da DataReader’a verilen değişken adı neyse, o olarak kullanma imkanı verir. Demek ki her bir satır, **rd** olarak ele alınmaktadır. Satır içerisindeki kolonları almak içinse; yukarıda bahsedildiği gibi bir kaç yol ve dikkat edilmesi gereken bazı noktalar vardır.

Bu şekilde verileri uygulama tarafına aldıktan sonra, uygulama tarafından veritabanına veri gönderme, güncelleme ve silme işlemlerinin nasıl yapılabileceğine değinilebilir. Ancak öncesinde SqlCommand, daha genel bir ifade olarak Command sınıfına ait **ExecuteScalar()** metodu için de, küçük bir örnek yapmak doğru olacaktır.

Bu örnekte, kayıtlı KATILIMCI sayısının alınacağı bir senaryomuz olsun. Temel olarak yapılması gereken işlemler yine aynıdır. SqlConnection nesnesini hazırladıktan sonraki kısmı tekrar yazacak olursak;

```
SqlConnection baglanti = new SqlConnection();
//baglanti nesnesini oluşturun, Constructor'ın aşırı yüklenmiş
//versiyonları da kullanılabilir.
baglanti.ConnectionString = "Data Source=.;Initial Catalog=SDS;
Integrated Security=True";
SqlCommand cmd = new SqlCommand();
//Command nesnesi oluşturulurken, Constructor'lardan
//yararlanılabilir.
//cmd nesnesinin çalıştıracağı SQL cümlesi yazılıyor.
cmd.CommandText = "SELECT COUNT(*) FROM KATILIMCI";
//cmd nesnesinin kullanacağı yani gideceği yeri gösteren connection
//nesnesi bağlanıyor.
cmd.Connection = baglanti;
//bağlantı açılıyor. Böylece veritabanı kullanıma açılmış oldu.
baglanti.Open();
int sayi=(int)cmd.ExecuteScalar();
//bağlantı kapatılıyor. Böylece veritabanı kullanıma kapatılmış
//oluyor ya da başka kullanıcılar için hazır hale geliyor.
baglanti.Close();
```

Kodlardan da anlaşılacağı gibi, geriye sadece bir tane sayı değeri gelecektir. Kayıt sayısı bir tam sayı olacaktır. Ancak Command'ın **ExecuteScalar()** metodu geriye **Object** tipinde bir değer dönmektedir. Bu durumda Cast işlemi yapmak gerekeceğini unutmamak gerekir.

Şimdi de sırasıyla yeni kayıt ekleme, kayıt güncelleme ve kayıt silme işlemlerini gösteren örnekleri inceleyelim.

Yapılacak işlemler yine aynıdır. SqlConnection nesnesi oluşturulacak, ve bu bağlantı üzerinde, asıl işlemleri yapan SqlCommand nesnesinde değişiklikler yapılacaktır.

Kayıt Ekleme

```
SqlConnection baglanti = new SqlConnection();
baglanti.ConnectionString = "Data Source=.;Initial Catalog=SDS;
Integrated Security=True";
SqlCommand cmd = new SqlCommand();
```

```

cmd.CommandText = "INSERT INTO KATILIMCI
KatIsim,KatSoyIsim,KatDogTar) VALUES ('Selçuk','UZUN','05.05.2001')";
cmd.Connection = baglanti;
baglanti.Open();
int etkilenenKayitSayisi=cmd.ExecuteNonQuery();
Console.WriteLine(etkilenenKayitSayisi + " kayıt girildi.");
baglanti.Close();

```

Bu örnek için girilen veriler, bu şekilde elle yazılmak yerine kullanıcıdan ya da bir arayüzden de alınabilir. Bu bir windows uygulaması, web uygulaması ya da mobil uygulama olabilir. Ancak verinin veritabanına gönderilmesi için mantık aynıdır. Sadece veriler dinamik bir şekilde dışarıdan alınmalıdır.

Kayıt Güncelleme

```

SqlConnection baglanti = new SqlConnection();
baglanti.ConnectionString = "Data Source=.;Initial Catalog=SDS;
Integrated Security=True";
SqlCommand cmd = new SqlCommand();
cmd.CommandText = "UPDATE KATILIMCI SET (KatIsim='Değişen
isim',KatSoyIsim='Değişen Soyisim' WHERE KatID=1)"
cmd.Connection = baglanti;
baglanti.Open();
int etkilenenKayitSayisi=cmd.ExecuteNonQuery();
Console.WriteLine(etkilenenKayitSayisi + " kayıt güncellendi.");
baglanti.Close();

```

Bu işlemde de kullanılan değerler, dinamik olarak dışarıdan alınabilmektedir.

Kayıt Silme

```

SqlConnection baglanti = new SqlConnection();
baglanti.ConnectionString = "Data Source=.;Initial Catalog=SDS;
Integrated Security=True";
SqlCommand cmd = new SqlCommand();
cmd.CommandText = "DELETE FROM KATILIMCI where
KatSoyIsim='UZUN'";
cmd.Connection = baglanti;
baglanti.Open();
int etkilenenKayitSayisi=cmd.ExecuteNonQuery();
Console.WriteLine(etkilenenKayitSayisi + " kayıt silindi.");
baglanti.Close();

```

Tüm bu işlemler sonucunda görülüyor ki; veritabanına veri eklemek, verileri güncellemek ya da veri silmek için gerekli olan temel bileşenler aynı, değişkenler sadece veritabanı yönetim sisteminin bu işlemleri yapması için verilen sql komutlarıdır.

Bu işlemlerde dikkat edersek, **ExecuteNonQuery()** metodu kullanılmaktadır. Geriye herhangi bir sonuç getirmemesine rağmen, alınmak zorunda olunmayan ve bu işlemten etkilenen kayıt sayısını veren bir metoddur.

Tüm bu işlemlerin ardından veritabanındaki değerler aşağıdaki son halini almıştır.

	KatID	KatIsim	KatSoyIsim	KatDogTar
►	1	Değişen isim	Değişen Soyisim	01.01.2007 00:...
	2	Burak	ALTUNTAŞ	02.02.2006 00:...
	3	Bülent	SÖZGE	03.03.2005 00:...
	4	Özgür	ALTUNTAŞ	NULL
*	NULL	NULL	NULL	NULL

Silme işlemi sonucu tablonun son durumu

Böylece Connected (Bağlantılı) modelle ilgili temel bilgileri ve bu temel bilgilerin kullanıldığı örnekler tamamlanmış oldu.

Sırada **Disconnected (Bağlantısız)** katman hakkında temel bilgilerin ve örneklerin olduğu yeni bir bölüm var.

EĞİTİM :

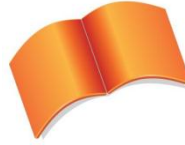
ADO.NET

Bölüm :

**Bağlantılı (CONNECTED)
Model & Bağlantısız
(DISCONNECTED) Model**

Konu :

Bağlantısız Modele Giriş



Microsoft Türkiye

Açık Akademi

Bağlantısız model ilk duyulduğunda, veritabanı ile uygulama arasında bir bağlantı yokmuş gibi bir fikir oluşturabilir. Ancak hemen belirtmekte yarar var ki, bağlantısız modelde de yine uygulama ile aynı makinada ya da uzak (Remote) bir makinada bir veritabanı, yine bir uygulama ve aralarında mutlaka ve mutlaka bir bağlantı olmak zorunda. Bu bağlantı kablo bağlantısı, internet bağlantısı ya da aynı makinadalar ise; birbirlerini görebilen uygulamalar arasındaki bağlantı olabilir. Peki nedir o zaman bu modelin bağlantısız olma özelliği?

Bu modelin bağlantısız olma özelliği, DataReader nesnesinde olduğu gibi ve o nesnenin en büyük sıkıntısı olan read-only, forward-only olması ve en önemlisi de **son satırı okuyana kadar, bağlantının açık kalması gerekliliğinin**, bu modelde olmayışıdır. Tabiki bu modelin, bu şekilde çalışabilmesini sağlamak için de farklı özelliklere sahip, çalışma mantığı farklı olan sınıflar yazılmıştır.

Bu bölümde, o yazılmış sınıflar ve nasıl çalıştıkları incelenecektir. Ancak öncesinde bağlantısız katmanın çalışma modelinden bahsedelim.

Günümüzde bir çok uygulamada kullanılan model budur. Sabah iş yerine gelen ya da bir şekilde veritabanına bağlanan kişiler, veritabanından verileri alırlar ve akşama kadar o verilerle çalıştıktan sonra, bir şekilde veritabanına tekrar bağlantı kurup verileri veritabanına gönderirler. Bu senaryoya en güzel örnekler plasiyerlerin kullandığı, ecza deposu yetkililerinin kullandığı uygulamalardır. Mobil uygulamalar genellikle bu modelle çalışırlar.

Bu modelde, veritabanıyla bağlantı kurulur. Ardından istenen verileri getirecek olan sql komut cümlesi yazılır. Bu cümleleri taşıyacak olan ve dönecek verileri tutacak olan nesneler yardımıyla veriler belleğe alındıktan sonra, veritabanıyla uygulama arasındaki bağlantı kapatılabilir. Ardından bellekteki veriler üzerinde istenilen şekilde çalışılır. İstenen veri kümesinin, istenen satırının, istenen kolonundaki değerine erişilebilir, değeri güncellenebilir ve hatta istenen satırlar silinebilir. Ancak bu modelin artılarının yanında, verilerin güncelliğini kaybetmesi, verilerin tutarlı olmaması gibi eksi yönleride vardır.

Şimdi de sırasıyla bu model için gerekli olan temel bileşenleri inceleyelim. Bu modelde de SqlConnection nesnesi aynı görevleri üstlenmiştir. O nedenle tekrar incelemek gerekmemektedir.

EĞİTİM :

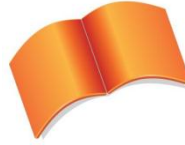
ADO.NET

Bölüm :

**Bağlantılı (CONNECTED)
Model & Bağlantısız
(DISCONNECTED) Model**

Konu :

SqlDataAdapter Nesnesi



Microsoft Türkiye

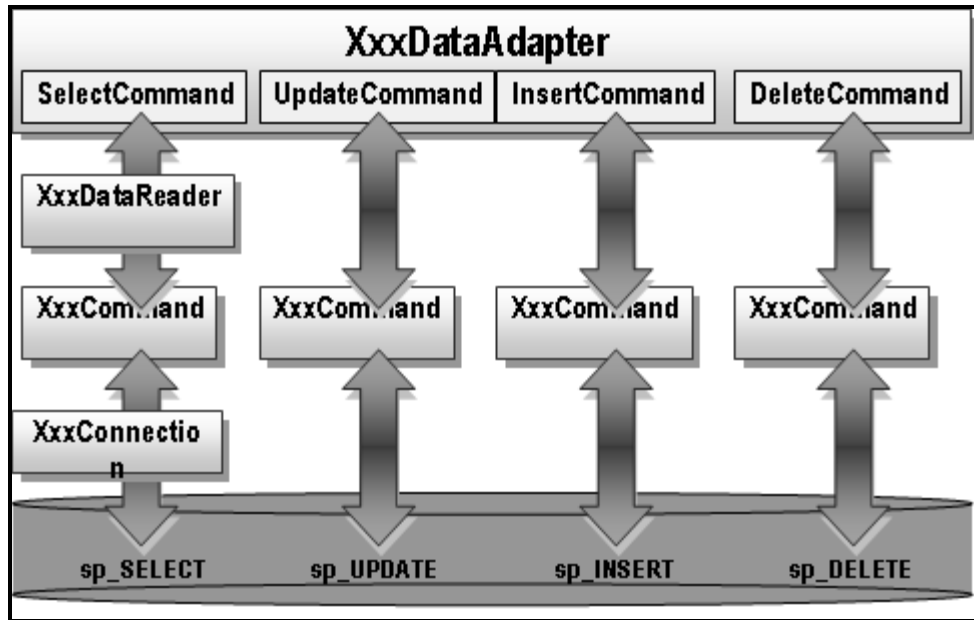
Açık Akademi

SqlDataAdapter

SqlDataAdapter class'ı bağlantısız modelin en güçlü bileşenidir. Bu nesne ile veritabanına bağlantı kurulduktan sonra yazılan sql cümlesini çalıştırıp, dönecek olan sonuç kümesini, bu sonuçları bellekte tutacak olan nesnelere yükleme işlemi kolayca yapılmaktadır. Bu işlem aslında veri getirme işlemi olarak düşünülmektedir. Ancak SqlDataAdapter sınıfı sadece veri getirmek için kullanılmaz, getirdiği bu veri üzerindeki değişiklikleri, yeni eklenen satırları, ve bellekte bulunan veri üzerinden silinen satırları tekrar veritabanına yansıtmak için kullanılan bileşen de budur.

Bu işlemleri yapabilmesi için, bu bileşende, Select (Seçme) işlemleri için **SelectCommand**, insert (ekleme) işlemleri için **InsertCommand**, delete (silme) işlemleri için **DeleteCommand** ve update (güncelleme) işlemleri için **UpdateCommand** adı verilen 4 farklı Command vardır. Bu command'lar belirli işlemleri yapmak için özelleşmişlerdir.

Burada bu command'lardan SelectCommand'i inceleyeceğiz. Bu command'lardan herbiri, bağlantılı modelde incelenen SqlCommand'ın özelliklerini aynen taşımaktadır.



DataAdapter Mimarisi

SelectCommand olarak bağlanan Command bir “select” cümlesi taşımalı ve DataAdapter daha sonra bu select cümlesinden gelecek sonucu bir yereye yükleyebilmelidir. Bu yükleme işlemi DataAdapter, **Fill()** adı verilen metodu ile yapmaktadır.

Yine bağlantılı modelde dikkat edilmesi gereken noktalardan bir tanesi bağlantının açılması ve kapatılmasıdır. Bu modelde ise; bağlantının açılması ve kapatılması işlemleriyle DataAdapter kendisi ilgilenmekte ve ihtiyaç duyduğunda yani verinin gelmesi istendiği anda bağlantıyı açıp, işlem tamamlandığında da kapatmaktadır.

Ayrıca SqlDataAdapter'in **Update()** adında yine çok önemli bir metodu daha vardır. Bu metod burada incelenmemektedir. Ancak kısaca bahsetmek gerekirse; bu metod aracılığıyla, parametre olarak verilen bellekteki bir tablo, daha önce ayarlanmış insert, update ve delete command'lara göre veritabanına

yansıtılmaktadır. Ayrıca unutulmamalıdır ki, DataAdapter sınıfı da provider'a göre özelleşmiştir ve varsayılan command'ı, **SelectCommand**'dir.

```
SqlConnection baglanti = new SqlConnection();
baglanti.ConnectionString = "Data Source=.;Integrated Security=True;
initial catalog=SDS";
SqlDataAdapter adaptor = new SqlDataAdapter();
SqlCommand cmd = new SqlCommand("SELECT * FROM KATILIMCI",baglanti);
adaptor.SelectCommand = cmd;
DataSet ds = new DataSet();
adaptor.Fill(ds);
```

Yukarıdaki kod bloğunda da görüldüğü gibi, adaptor adında bir nesne oluşturulmuştur ve bu nesnenin SelectCommand adındaki özelliğine, select işlemi yapan ve sonuc kümesi veren bir SqlCommand bağlanmıştır. Ardından bu işlemlerden çıkacak sonuçları tutmak için kullanılacak **DataSet** adındaki diğer bir sınıftan nesne oluşturulmuştur ve bu nesneye gelen verileri doldurması için adaptor nesnesinin **Fill()** metodu kullanılmıştır. Peki buradaki DataSet sınıfı nedir ve bu sınıfın özellikleri nelerdir?

EĞİTİM :

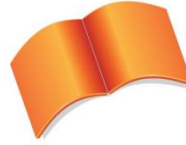
ADO.NET

Bölüm :

**Bağlantılı (CONNECTED)
Model & Bağlantısız
(DISCONNECTED) Model**

Konu :

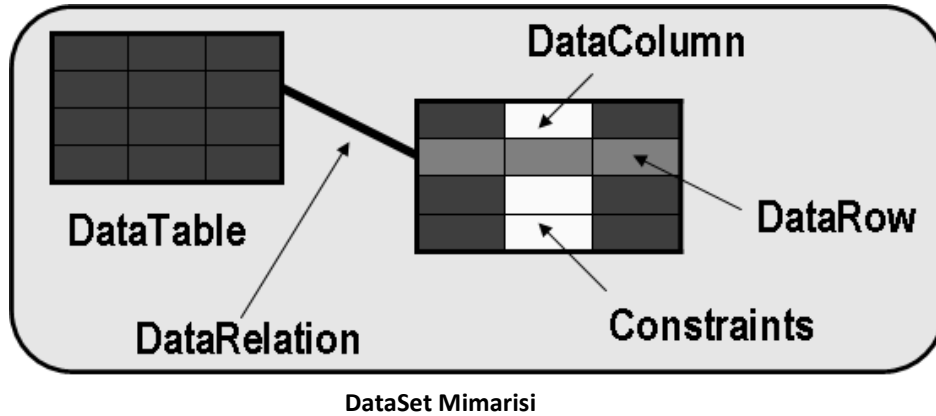
DataSet Nesnesi



Microsoft Türkiye

Açık Akademi

DataSet



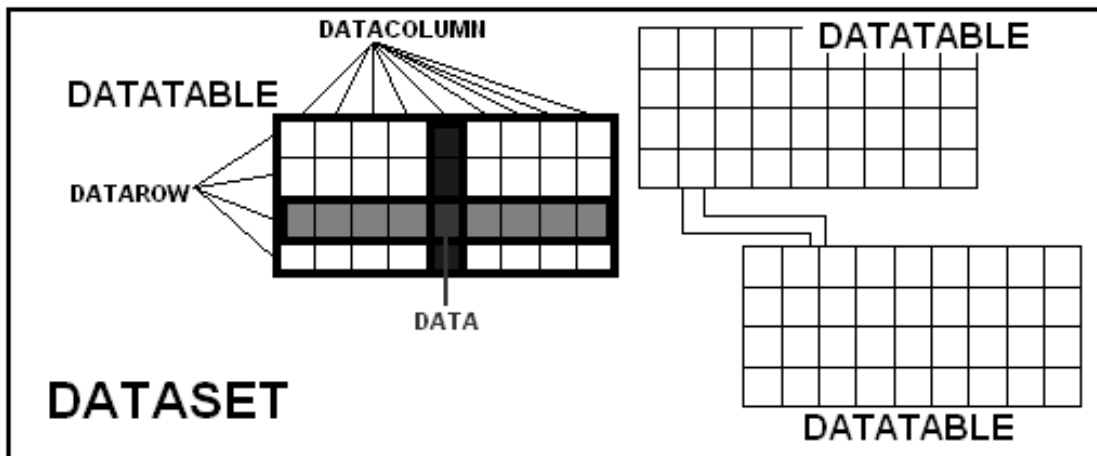
Yukarıdaki şekilden de anlaşılacağı üzere DataSet, aslında veritabanının uygulama tarafındaki versiyonu olarak düşünülebilir. Veritabanının bire-bir kopyası olarak kullanılabilecek DataSet, uygulamanın çalıştığı makinanın belleğine yerleşir ve orada sanki veritabanıymış gibi kullanılabilir. Mesela bir DataSet üzerinde sorgulamalar, güncellemeler, eklemeler ve kayıt silme işlemleri yapılabilmektedir. Tabii ki şunu da karıştırmamak gerekir; Bir DataSet veri tutmaz, veriyi DataSet içerisindeki **DataTable** adı verilen nesneler tutmaktadır. Bu nedenle, DataAdapter class'ının **Fill()** metoduna da DataSet'i doldurması söylendiğinde, aslında DataSet'in içerisine bir tane DataTable açar ve onun içerisine verileri atar. Ayrıca bir DataSet içerisinde birden çok DataTable olabilir ve bu tablolar arasında **DataRelation** adı verilen bağlantı nesneleri yer almaktadır.

DataTable nesnesi de aslında içerisinde bir çok bileşen bulundurmaktadır. Bunlardan en önemlisi ve tablonun yapısını belirten **DataColumn** sınıfıdır. Bu sınıfla, kolonun yapısı, özellikleri, adı, tipi, alabileceği verinin büyüklüğü vb. bilgiler tutulmaktadır. Diğer önemli bir bileşen de **DataRow**'dur. DataTable içerisindeki her bir satır, DataRow olarak belirtilmektedir. Veriler, DataColumn ve DataRow'ların kesiştiği **hücre**lerde tutulmaktadır.

Demek ki bir tek değere erişmek için geçilmesi gereken noktalar şu şekilde özetlenebilir.

DataSet'in tablolarından ilgili tablo, bu tablonun satırlarından ilgili satır, bu satırın ilgili kolonu şeklinde ilerlemek gerekmektedir.

Bir DataSet'in tabloları bir koleksiyon, bir tablonun satırları ve sütunları da birer koleksiyondur. Herşey listeler üzerinden çalışmaktadır.



DataSet Mimarisi

Bir dataset'i kodlarken;

Öncelikle DataSet class'ının bulunduğu **System.Data** isim alanına giriş yapılmalıdır. Bu isim alanı altında, DataSet, DataTable, DataColumn, DataRow yani provider'dan bağımsız, temel ADO.NET bileşenleri yer almaktadır.

```
//ds adında, DataSet class'ından bir nesne oluşturuluyor. DataSet'in adı ise; DSKATILIMCILAR.  
//Buradaki DSKATILIMCILAR ismi, veritabanı ismi olarak düşünülebilir.  
DataSet ds = new DataSet("DSKATILIMCILAR");  
  
//ds'nin tablolarından ilkinin, ilk satırının, ilk sütunundaki değeri verir.  
ds.Tables[0].Rows[0][0].ToString();  
  
//ds'tablolarından "KATILIMCI" tablosunun 6. sıradaki satırının, "KatIsim" kolonuna karşılık gelen değeri verir.  
ds.Tables["KATILIMCI"].Rows[5]["KatIsim"].ToString();  
  
//DataTable class'ından bir nesne oluşturuluyor. Adı "KATILIMCI2". Bu isim, dataset'in tablolarından ilgili datatable'a gitmek gerektiğinde kullanılabilir.  
DataTable dt = new DataTable("KATILIMCI2");  
  
//Oluşturulan DataTable nesnesi, bir DataSet nesnesinin tablo koleksiyonuna ekleniyor.  
ds.Tables.Add(dt);  
  
//Ds'nin tablolarından yukarıda eklenen "KATILIMCI2" tablosunun satırlarına yeni bir DataRow ekleniyor.  
ds.Tables["KATILIMCI2"].Rows.Add(yenisatir);  
  
//Ds'nin tablolarından aslında yukarıda eklenen "KATILIMCI2" tablosuna, yani 2. tablosunun kolonlarına yeni bir DataColumn ekleniyor.  
ds.Tables[1].Columns.Add(kolonadi);
```

Yukarıda görülen kodlarda, bir DataSet'e alınan veriye nasıl erişilebileceği konusunda farklı yöntemler görülmektedir.

Peki bu DataSet herhangi bir veri kaynağından gelen verilerle nasıl doldurulur?

Bu noktada, yukarıda sayılan bağlantısız model bileşenleri devreye girmektedir. Yani öncelikle SqlConnection ardından SqlDataAdapter ve onun SelectCommand'ı, oluşturulmalı ve tabii ki bu command'ın CommandText'i yazılmalıdır. Aslında sadece bunların oluşturulması yeterli olacaktır. Sonrasında, SqlDataAdapter'in **Fill()** metoduna, içine verilerin doldurulması istenen, DataTable nesnesini ya da DataSet nesnesini parametre olarak vermek, verilerin veritabanından uygulamaya taşınması için yeterli olacaktır.

```
SqlConnection baglanti = new SqlConnection("Data source=.;initial catalog=SDS;integrated security=True");  
  
//SqlDataAdapter'ın varsayılan Command'ı, SelectCommand'dır. O nedenle Constructor aracılığıyla SelectCommand'ın commandtext'i ve kullanacağı bağlantı bilgisi girilebilmektedir.  
SqlDataAdapter adaptor = new SqlDataAdapter("SELECT * FROM KATILIMCI",baglanti);
```

```

//DataTable nesnesi oluşturuluyor.
DataTable dt = new DataTable();

//adaptor aracılığıyla, dt tablosu gelecek olan verilerle
dolduruluyor.
adaptor.Fill(dt);

//DataSet nesnesi oluşturuluyor.
DataSet ds = new DataSet();
//ds'nin tablo listesine, veritabanından gelen verileri tutan dt de
ekleniyor. Aslında eklenmeden de tek başına kullanılabilir.
ds.Tables.Add(dt);

//ds'nin tablolarından ilki olan dt tablosundaki tüm isimler ekrana
yazdırılıyor.
for (int i = 0; i < ds.Tables[0].Rows.Count; i++)
{
    //ilk tablonun her bir satırı için bu blok çalışacaktır.
    Console.Write(ds.Tables[0].Rows[i]["KatIsim"].ToString());
    //Katılımcıların KatDogTar kolonundaki bilgilerine erişmek için
    //KatDogTar kolonu 3 nolu indeksteki kolondur.
    Console.Write("\t" + ds.Tables[0].Rows[i][3].ToString());
}

//sadece 5. katılımcı bilgileri alınabilir
ds.Tables[0].Rows[4]["KatID"].ToString();

//sadece 5. katılımcının soyisim bilgisini verir.
ds.Tables[0].Rows[4][2].ToString();

```

Bu örnek de gösteriyor ki; Bağlantısız katmanda, bağlantının açılması ya da kapatılması ile uğraşmaya gerek kalmaz. Ayrıca bağlantılı modelde olduğu gibi, istenilen veriye erişmek için, tüm kayıtları baştan tek tek geçmek gerekmemektedir. Direkt olarak istenilen veriye odaklanılabilmekte ve hatta değeri de değiştirilebilmektedir.

Sıradaki örnekte, Bağlantısız modeli anlayabilmek için gerekli olan en temel bileşenler ve nasıl kullanılacakları görülebilir.

EĞİTİM :

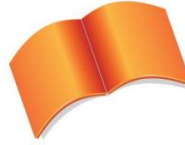
ADO.NET

Bölüm :

**Bağlantılı (CONNECTED)
Model & Bağlantısız
(DISCONNECTED) Model**

Konu :

**Bağlantısız Model – Örnek
Uygulama**



Microsoft Türkiye

Açık Akademi

Örnek Uygulama

Kullanılacak veritabanı daha önceki örnekte de kullanılan SDS'dir. O nedenle bağlantı cümlesi yine aynıdır, yani aynı SqlConnection nesnesi kullanılabilir.

Bağılantısız katmanın en güçlü ve olmazsa olmaz bileşeni olan SqlDataAdapter sınıfından bir tane nesne oluşturduktan sonra, o nesnenin SelectCommand özelliği için de bir tane SqlCommand nesnesi oluşturulmalı, CommandText ve Connection özellikleri ayarlanmalıdır. Ardından sadece SqlDataAdapter nesnesinin **Fill()** metodunu kullanarak gelecek olan verileri, ya DataTable nesnesine ya da DataSet nesnesine doldurup kullanmak kalmaktadır.

Unutulmamalıdır ki, bağlantısız katmanın bileşenlerinden DataSet, DataTable, DataRow ve DataColumn sınıfları, provider bağımsız ve **System.Data** isim alanı içindedir. Ancak veritabanına bağlanmak ve hangi verilerin istendiğini söylemek için ve son olarak da bu verileri getirip ilgili nesnelere doldurmak için kullanılan, Connection, Command ve DataAdapter bileşenleri ise; provider'a özel olarak kullanılır. Bu örnekte, SQL Server .NET Data Provider kullanılarak ilerleyeceğiz.

```
using System.Data.SqlClient;
using System.Data;
.....
.....

SqlConnection baglanti = new SqlConnection("Data source=.;initial
catalog=SDS;integrated security=True");

//SqlDataAdapter'ın varsayılan Command'ı, SelectCommand'dır. O
nedenle Constructor aracılığıyla SelectCommand'ın commandtext'i ve
kullanacağı bağlantı bilgisi girilebilmektedir.
SqlDataAdapter adaptor = new SqlDataAdapter("SELECT * FROM
KATILIMCI",baglanti);

//DataTable nesnesi oluşturuluyor.
DataTable dt = new DataTable();

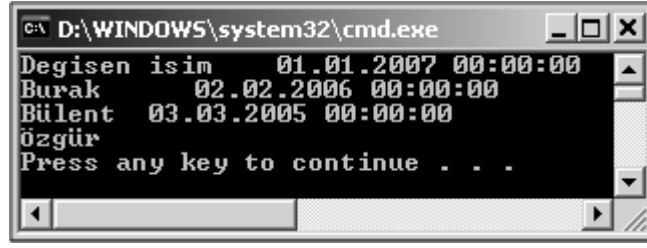
//adaptor aracılığıyla, dt tablosu gelecek olan verilerle
dolduruluyor.
adaptor.Fill(dt);

//DataSet nesnesi oluşturuluyor.
DataSet ds = new DataSet();

//ds'nin tablo listesine, veritabanından gelen verileri tutan dt de
ekleniyor. Aslında eklenmeden de tek başına kullanılabilmektedir.
ds.Tables.Add(dt);

//ds'nin tablolarından ilki olan dt tablosundaki tüm isimleri ekrana
yazdırılıyor.
for (int i = 0; i < ds.Tables[0].Rows.Count; i++)
{
    //ilk tablonun her bir satırı için bu blok çalışacaktır.
    Console.Write(ds.Tables[0].Rows[i]["KatIsim"].ToString());
    //Katılımcıların KatDogTar kolonundaki bilgilerine erişmek için
    //KatDogTar kolonu 3 nolu indeksteki kolondur.
    Console.Write("\t" + ds.Tables[0].Rows[i][3].ToString()+ "\n");
}
```

Bu örneğin çalıştırılması sonucunda aşağıdaki çıktı oluşmaktadır.



```
C:\ D:\WINDOWS\system32\cmd.exe
Degisen isim 01.01.2007 00:00:00
Burak 02.02.2006 00:00:00
Bulent 03.03.2005 00:00:00
Ozgür
Press any key to continue . . .
```

DataTable içerisindeki satırların elde edilmesi

Tabii ki bu konuda farklı örnekler verilebilir. Ancak senaryo hep aynı olacaktır. Değişen şey, sadece bağlantı cümlesi, ilgili veritabanına göre Provider ve bileşenler, istenen veriyi almak için de, Sql cümlesinin değişmesi yeterli olacaktır. Ardından hangi bilgilere ihtiyaç varsa; DataSet'in tablolarından ilgili tablonun ilgili satır ve sütunlarından veriler alınabilmektedir.